

Cycling Ontology Report

Author

Samuel GÓMEZ HINCAPIÉ

samuel.gomez-hincapie@eurecom.fr

In this report, I explain how I created my first ontology model `GomezHincapie-cycling.owl`. To do so I used [Protégé](#) version 5.6.9 to model the ontology and the [HermiT](#) reasoner version 1.4.3.456 to check its coherence and consistency.

In the interest of full transparency, **this report was 100% written by a human: me**. This includes the design, the content and the structure of the report. That said, I used AI (DeepSeek V4.1 Flash) to improve some parts of the writing and to help me understand the assignment, as this is my first time working with an ontology and with this software.

Questions

As indicated in the [assignment](#), this report contains the answers to questions 1, 2, 5, 6, 7, 9, 11 and 12.

1. Prefix and URI

Question

Start Protégé and imagine a good **namespace prefix** and a **namespace URI** for modeling the Cycling domain. Provide this couple prefix/uri in your report and justify why you choose them.

In order to keep my own domain, where I could define any entity, I chose to create a repository on [GitHub](#) for this class and, inside it, a folder for this project `~/WEBSEM/cycling`. Taking that domain as the ontology IRI, I created the prefix `cyc:` because it is explicit enough.

Item	Value
Prefix name	<code>cyc:</code>
Prefix (namespace)	https://github.com/Samuel2165/WEBSEM/cycling#
Ontology IRI	https://github.com/Samuel2165/WEBSEM/cycling

The `#` belongs to the namespace of the entities (`cyc:Race` , `cyc:Stage` , ...), whereas the ontology IRI identifies the ontology document itself and is therefore written without it.

2. Classes

? Question

In the cycling domain, a `Race` is either a `OneDayRace` or a `SeveralStagesRace`. There are no other types of race. Model these three classes in the ontology and write in the report the logical formula equivalent to this definition.

The three classes are related by a single **disjoint union** axiom, which states both the closure ("no other types of race") and the mutual exclusion of the two sub-types:

$$\text{Race} \equiv \text{OneDayRace} \sqcup \text{SeveralStagesRace}$$

equivalently `DisjointUnion(Race, OneDayRace, SeveralStagesRace)`, which unfolds into `Race  $\equiv$  OneDayRace  $\sqcup$  SeveralStagesRace` together with `OneDayRace  $\sqsubseteq$   $\neg$ SeveralStagesRace`. The `$\equiv$` (rather than a plain `$\sqsubseteq$` on each sub-type) is what rules out any further type of race. In Protégé this is the `Disjoint Union Of` row of the `Race` class description.

The serialisation of that axiom in the exported ontology (`GomezHincapie-cycling.owl`, RDF/XML syntax) is the following (the namespace declarations introduced by the FOAF import are omitted, and the axioms of `SeveralStagesRace` that belong to question 3 are not repeated here):

```
<rdf:RDF xmlns="https://github.com/Samuel2165/WEBSEM/cycling#"
  xml:base="https://github.com/Samuel2165/WEBSEM/cycling"
  xmlns:cyc="https://github.com/Samuel2165/WEBSEM/cycling#"
  xmlns:owl="http://www.w3.org/2002/07/owl#"
  xmlns:rdf="http://www.w3.org/1999/02/22-rdf-syntax-ns#"
  xmlns:rdfs="http://www.w3.org/2000/01/rdf-schema#">

  <owl:Ontology rdf:about="https://github.com/Samuel2165/WEBSEM/cycling">
    <owl:imports rdf:resource="http://xmlns.com/foaf/0.1/" />
  </owl:Ontology>

  <!-- https://github.com/Samuel2165/WEBSEM/cycling#Race -->
  <owl:Class
    rdf:about="https://github.com/Samuel2165/WEBSEM/cycling#Race">
    <owl:disjointUnionOf rdf:parseType="Collection">
      <rdf:Description
        rdf:about="https://github.com/Samuel2165/WEBSEM/cycling#OneDayRace"/>
```

```

    <rdf:Description
rdf:about="https://github.com/Samuel2165/WEBSEM/cycling#SeveralStagesRace"/>
    </owl:disjointUnionOf>
</owl:Class>

<!-- https://github.com/Samuel2165/WEBSEM/cycling#OneDayRace -->
<owl:Class
rdf:about="https://github.com/Samuel2165/WEBSEM/cycling#OneDayRace">
    <rdfs:subClassOf
rdf:resource="https://github.com/Samuel2165/WEBSEM/cycling#Race"/>
    </owl:Class>

<!-- https://github.com/Samuel2165/WEBSEM/cycling#SeveralStagesRace -->
<owl:Class
rdf:about="https://github.com/Samuel2165/WEBSEM/cycling#SeveralStagesRace">
    <rdfs:subClassOf
rdf:resource="https://github.com/Samuel2165/WEBSEM/cycling#Race"/>
    </owl:Class>
</rdf:RDF>

```

5. Prologue

? Question

Define with necessary and sufficient conditions (i.e. strict equality) the `Prologue` class such as the `Prologue` is exactly always the first Stage of a `SeveralStagesRace`. You might have to introduce more properties or use the inverse of an existing property to do this modeling. Write in the report the logical formula equivalent to this definition. Hint: in Protégé, you can use the keyword `value`.

The `Prologue` class is defined by necessary and sufficient conditions, i.e. by strict equality:

$$\text{Prologue} \equiv \text{Stage} \sqcap \exists \text{isFirstStageOf. SeveralStagesRace}$$

together with these two property axioms:

$$\text{hasFirstStage} \equiv \text{isFirstStageOf}^{-1}$$

$$\text{FunctionalObjectProperty}(\text{hasFirstStage})$$

The property `composedOf` (from question 3) relates a race to its stages but carries no ordering, and neither does its inverse: every stage is a stage of some race, so an equivalence built on `composedOf-1` alone would classify *all* stages as prologues. I therefore introduced `hasFirstStage: SeveralStagesRace → Stage` to name the notion of "first stage" and declared it functional, so that a race has exactly one first stage, and used its inverse

`isFirstStageOf` to state the condition from the stage's side. The equivalence supplies both directions: every prologue is the first stage of some several-stages race, and every stage that is the first stage of a several-stages race is a prologue. Written as a one-way subclass axiom the class would remain primitive and the reasoner would infer nothing; with $\equiv$ `Hermit` classifies any stage asserted as the first stage of a race as a `Prologue`.

A final note about the hint in the statement. The `value` keyword is the Manchester syntax for a has-value restriction, so `stageNumber value 1` is what Protégé writes for $\exists \text{stageNumber. } \{1\}$, and it fits a model in which every stage carries its position number:

$$\text{Prologue} \equiv \text{Stage} \sqcap \exists \text{stageNumber. } \{1\}$$

with `stageNumber: Stage  $\rightarrow$  xsd:integer`. I did not take that route because a position number tells which place a stage holds, but not which race it belongs to, whereas the definition I used keeps the link to the `SeveralStagesRace` that the exercise asks about. I mention it because it is the reading that the hint points at, and it would also be a correct way of modelling the first stage of a race.

6. The Person hierarchy

? Question

Add the concept `Person` to the ontology. A `Person` can be a `Spectator`, a `RacePerson` or a `TeamPerson` among others. A `TeamPerson` is exactly the union of a `Doctor`, a `Director` or a `RaceCyclist`. A `RaceCyclist` can himself be considered as a `Rider`, a `Sprinter` or a `Climber`. A `Person` has a `name`, an `age`, and a `nationality`. A `Person` participates in `Race`. Add all these classes and properties and the corresponding axioms. Explain in the report what are the differences between object and datatype properties.

a. Constraints

These are the axioms I added for this question. I grouped them by how strong the relation is, because the wording of the assignment changes from one class to the next and the axioms have to follow it.

For the three sub-classes of `Person` I only declared subsumption, because the assignment says that a `Person` can be one of them **among others**, so `Person` has to stay open:

$$\text{Spectator} \sqsubseteq \text{Person} \quad \text{RacePerson} \sqsubseteq \text{Person} \quad \text{TeamPerson} \sqsubseteq \text{Person}$$

`TeamPerson` is the exception, because here the assignment says **exactly** the union. That one is an equivalence together with disjointness, which Protégé writes as a single `Disjoint Union`

Of axiom:

$$\text{TeamPerson} \equiv \text{Doctor} \sqcup \text{Director} \sqcup \text{RaceCyclist}$$

The three kinds of cyclist are sub-classes again, and I did **not** make them disjoint from each other, because in cycling a rider can be both a sprinter and a climber:

$$\text{Rider} \sqsubseteq \text{RaceCyclist} \quad \text{Sprinter} \sqsubseteq \text{RaceCyclist} \quad \text{Climber} \sqsubseteq \text{RaceCyclist}$$

These are the properties, the three datatype properties first and the participation relation last:

$$\begin{aligned} \text{name} : \text{Person} &\rightarrow \text{xsd:string} & \text{age} : \text{Person} &\rightarrow \text{xsd:integer} & \text{nationality} : \text{Person} &\rightarrow \text{xsd:string} \\ \text{Person} &\sqsubseteq \exists \text{name.xsd:string} & \text{Person} &\sqsubseteq \exists \text{age.xsd:integer} & \text{Person} &\sqsubseteq \exists \text{nationality.xsd:string} \\ \text{participatesIn} : \text{Person} &\rightarrow \text{Race} \end{aligned}$$

The difference between these groups is not cosmetic. `Person` and `RaceCyclist` are **primitive** classes, so the reasoner only knows that a spectator is a person, never the other way round, and it cannot classify anything as one of them on its own. `TeamPerson` is a **defined** class instead, so HermiT can infer that whoever is a doctor, a director or a race cyclist is a team person, and the disjointness tells it that it is exactly one of the three and not two of them at the same time.

In the exported ontology, that restriction is written like this:

```
<!-- https://github.com/Samuel2165/WEBSEM/cycling#TeamPerson -->
<rdf:RDF xmlns:owl="http://www.w3.org/2002/07/owl#"
  xmlns:rdf="http://www.w3.org/1999/02/22-rdf-syntax-ns#"
  xmlns:rdfs="http://www.w3.org/2000/01/rdf-schema#">
  <owl:Class
    rdf:about="https://github.com/Samuel2165/WEBSEM/cycling#TeamPerson">
    <rdfs:subClassOf
      rdf:resource="https://github.com/Samuel2165/WEBSEM/cycling#Person"/>
    <owl:disjointUnionOf rdf:parseType="Collection">
      <rdf:Description
        rdf:about="https://github.com/Samuel2165/WEBSEM/cycling#Director"/>
      <rdf:Description
        rdf:about="https://github.com/Samuel2165/WEBSEM/cycling#Doctor"/>
      <rdf:Description
        rdf:about="https://github.com/Samuel2165/WEBSEM/cycling#RaceCyclist"/>
    </owl:disjointUnionOf>
  </owl:Class>
</rdf:RDF>
```

`Doctor`, `Director` and `RaceCyclist` also appear as explicit sub-classes of `TeamPerson`, and their pairwise disjointness is asserted explicitly as well. Both things are redundant, because

the disjoint union already entails them, but they do not harm and they make the asserted hierarchy easier to read in the Protégé tree.

b. Properties

For the participation I added the object property `participatesIn`, with `Person` as domain and `Race` as range. I did not add a restriction forcing every `Person` to participate in some `Race`, because a `Spectator` does not participate in the same sense as a `RacePerson` and the statement does not make that distinction explicit, so I preferred to leave it as a plain relation rather than inventing a constraint that the exercise does not ask for.

And regarding the two kinds of property:

An **object property** relates two *individuals*: its domain and range are classes, and its filler in a restriction is another individual or an anonymous class expression (`composedOf some Stage`). Because it stays inside the individual domain it supports the expressive constructs: inverse properties, functional/inverse-functional, transitive, symmetric and asymmetric characteristics, and qualified cardinalities such as `hasMember exactly 10 RaceCyclist`.

A **datatype property** relates one individual to a *literal value* drawn from a datatype, typically from XSD (`name → xsd:string`, `age → xsd:integer`). It has no inverse (a literal is not an individual), it cannot be transitive or symmetric, and its restrictions are unqualified: you can write `age some xsd:integer` but not `age some Doctor`.

In OWL 2 these are two distinct kinds of entity that cannot be swapped: that is why aligning your `age` with `foaf:age` is a *datatype* property alignment, while aligning `belongsTo` with `foaf:member` is an *object* property alignment.

(In the OWL/RDF syntax both are `rdf:Property`; the OWL 2 syntax distinguishes `ObjectProperty` from `DataProperty`. Protégé keeps them in two separate tabs for that reason.)

7. Property `belongsTo`

Question

A `Team` is exactly composed of 1 `Doctor`, 1 `Director` and 10 `RaceCyclist`. Define this class using the `belongsTo` property. Explain in the report what can you say about this new property with respect to previously defined properties

Here, two new properties are needed to satisfy the constraints, as shown in the equations below:

$$\text{hasMember} \equiv \text{belongsTo}^{-1}$$

$\text{Team} \equiv (= 1 \text{ hasMember. Doctor}) \sqcap (= 1 \text{ hasMember. Director}) \sqcap (= 10 \text{ hasMember. RaceCyclist})$

We can say that, contrary to the other properties defined up to this point:

- This is the **first property that only makes sense through its inverse**. `composedOf` (`race` $\rightarrow$ `stage`) and `participatesIn` (`person` $\rightarrow$ `race`) can be read on their own; the cardinalities that define `Team` must be written on `hasMember`, the inverse of `belongsTo`, because the constraint is about what a team *contains*, not about what a person *is in*.
- It is the **first property used to define a class** (the $\equiv$ in `Team`), whereas the earlier properties only appeared in one-way $\sqsubseteq$ restrictions. A defined class lets the reasoner **classify**: give an individual the right 12 memberships and Hermit infers it is a `Team` even without being told.
- Like `composedOf` it is a **part/whole-style relation between two classes of individuals**, unlike `name / age / nationality` which are datatype properties pointing at literals.
- It is **not** `composedOf`: a race is composed of stages, a person belongs to a team. Reusing `composedOf` here would collapse two different relations into one and corrupt both.

9. Integration with [FOAF \(Friend of a friend\)](http://xmlns.com/foaf/spec/) Ontology

? Question

Load (import) the FOAF ontology using its <http://xmlns.com/foaf/spec/> and align some of the concepts you have defined such as: the class `Person` is equivalent to the class `Person` defined in FOAF and the class `Team` is a subclass of the class `Organisation` defined in FOAF. Search for properties in the FOAF ontology to align with your own properties and add the corresponding mapping axioms. Write in the report the mappings you have discovered.

I loaded FOAF in Protégé with the import wizard; the assignment gives the IRI <http://xmlns.com/foaf/spec/>, which serves the same RDF/XML document as the canonical <http://xmlns.com/foaf/0.1/> that Protégé stores in its import catalog. Once FOAF was imported, its entities became available for my own axioms, and I added the following mappings:

My entity	FOAF term	Axiom type	Note
<code>cyc:Person</code>	<code>foaf:Person</code>	<code>owl:equivalentClass</code>	asked for explicitly: both classes have exactly the same extension
<code>cyc:Team</code>	<code>foaf:Organization</code>	<code>rdfs:subClassOf</code>	asked for explicitly. Note the American spelling:

My entity	FOAF term	Axiom type	Note
			foaf:Organisation does not exist
cyc:belongsTo	foaf:member	owl:inverseOf	foaf:member goes from a group to an agent, so it is the inverse of belongsTo, which goes from a person to a team
cyc:name	foaf:name	rdfs:subPropertyOf	both are datatype properties
cyc:age	foaf:age	rdfs:subPropertyOf	both are datatype properties

I could not align `cyc:nationality`, because FOAF defines no nationality property. Its closest term is `foaf:based_near`, but that one points to a `wgs84:SpatialThing`, that is, to a place rather than to a literal, so it does not express the same relation.

Two things I discovered while doing this exercise, which I mention because they cost me some time. First, `foaf:name` and `foaf:age` are *datatype* properties, not object properties, so they do not appear in the `Object Properties` tab of Protégé; I had to look for them in the `Data Properties` tab (and `foaf:name` also shows up among the annotation properties, which confused me for a while). Second, FOAF itself declares `foaf:Person owl:disjointWith foaf:Organization`, and since my `cyc:Person` is equivalent to `foaf:Person` while my `cyc:Team` is a subclass of `foaf:Organization`, no entity in my ontology can ever be both a `Person` and a `Team`. That is what I intended and the reasoner confirms it.

11. Reusable classes and properties

Question

Use the LOV search function at <https://lov.linkeddata.es/dataset/terms/> and identify relevant classes and properties that could be re-used in your ontology. Write them down in your report.

After a quick search, these are some of the classes and properties that could be reused in my own ontology:

Search	Result	Returned IRI	W
cyclist	dbpedia-owl:Cyclist	http://dbpedia.org/ontology/Cyclist	th co R R
cyclist	dbpedia-owl:cyclistGenre	http://dbpedia.org/ontology/cyclistGenre	ca cy S Sl
team	dbpedia-owl:proTeam	http://dbpedia.org/ontology/proTeam	th te T
team	vivo:Team , dbpedia-owl:team	http://vivoweb.org/ontology/core#Team , http://dbpedia.org/ontology/team	al cl
nationality	dbpedia-owl:nationality , schema:nationality	http://dbpedia.org/ontology/nationality , http://schema.org/nationality	a c fc of
athlete	schema:athlete , dbpedia-owl:Athlete	http://schema.org/athlete , http://dbpedia.org/ontology/Athlete	re R

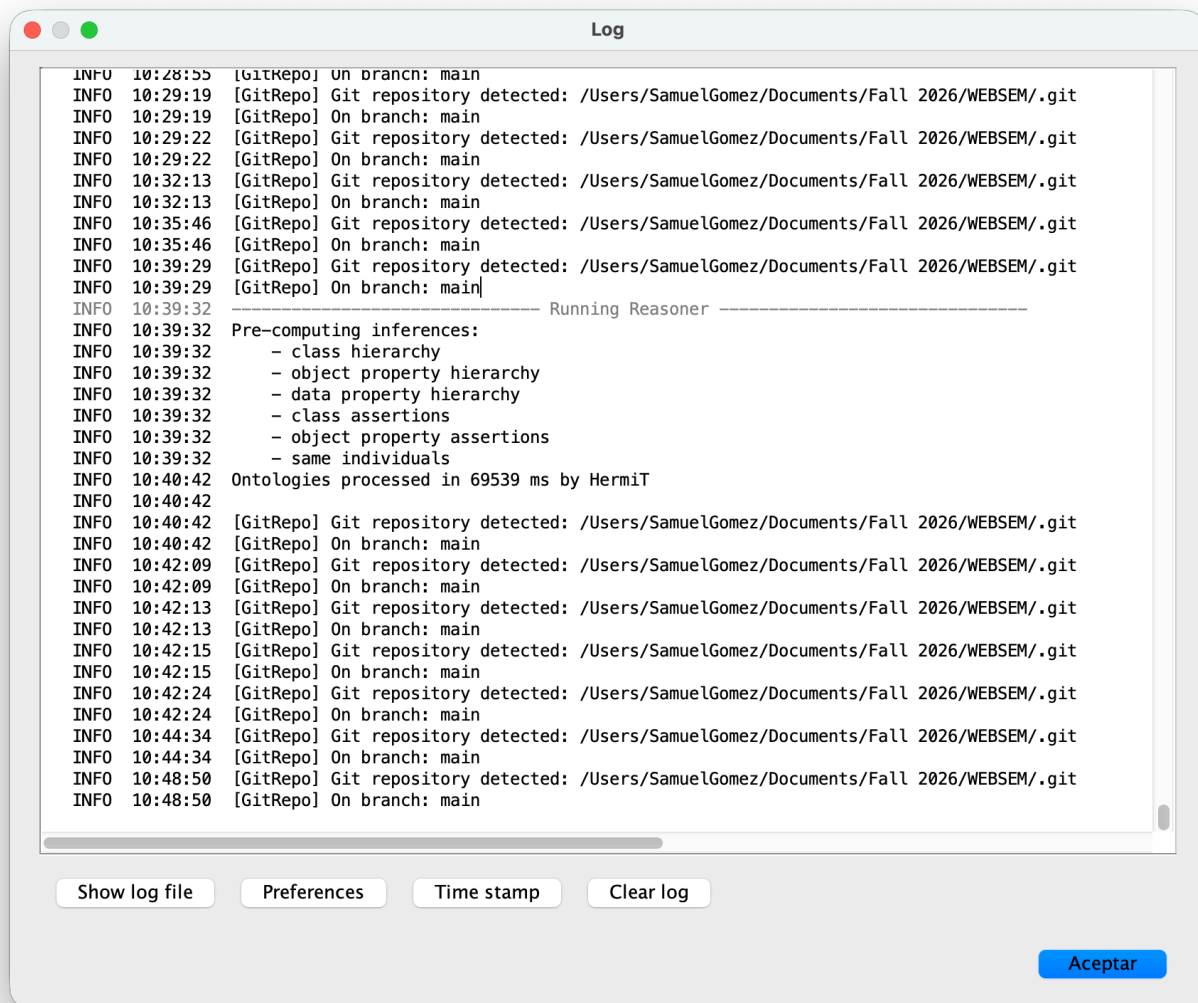
The mappings I would actually reuse are `dbpedia-owl:Cyclist` for `RaceCyclist` and `schema:nationality` for `cyc:nationality`. I would not import the whole DBpedia Ontology though, because the size of its import closure is disproportionate for an ontology this small, so a single aligned axiom per interesting term is the reasonable middle ground.

12. Validation and results

Question

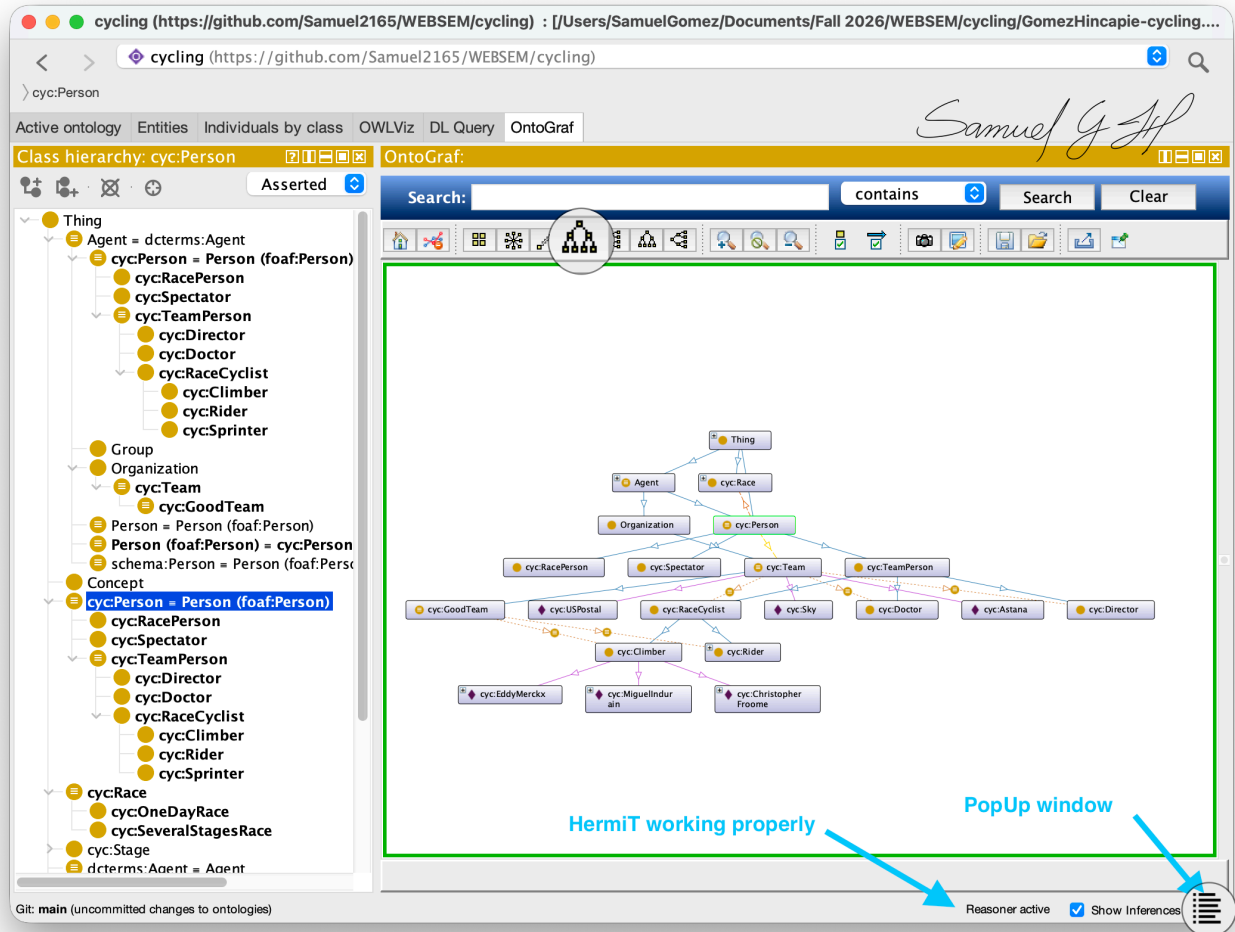
Validate your ontology by clicking on `Reasoner` and `Start reasoner` where `HermiT` is selected as the reasoning service. What is the result? Did you get the message “*your ontology is coherent and consistent*”

The ontology works properly with the reasoner. What I never get is the message “*your ontology is coherent and consistent*”: instead I get the message shown below, which appears in a pop-up window once the reasoner finishes.



The log lists the inference types that Hermit pre-computed and reports no warning and no error, and no class appears as unsatisfiable in the class hierarchy. That is the same result expressed in different words: my ontology is coherent and consistent, but the Protégé plug-in simply does not phrase it that way.

The next screenshot shows the ontology in the `OntoGraf` panel, which gives a clear visualization of the whole model once the reasoner is active.



Difficulties

As indicated, here are the main difficulties that I had while working on this assignment.

- The software, of course, was easier to learn than writing everything in `xml` in a code editor, which does not mean that I did not have to investigate how to use it, read manuals and watch some YouTube videos. It also crashed once; I did not lose my work, but it is a little bit buggy every time I open it.
- The mathematical notation for the logic is also new for me, at least some of its terms, so while I was working on the model I was learning what all the symbols meant, as well as the cardinality concepts.
- Even though I come from a very different background and did not know anything about the web or about what an ontology is, having all my notes in [Obsidian](#) made the concepts easier for me to understand.

Finally, I would like to say that this is a very interesting topic for me. I like to keep order in everything I have, and this is exactly the kind of tool that I use every day to understand how information is linked together.